Hermetic Computing: A Reading of Cryptographic and Computational Primitives Through the Seven Hermetic Principles

Juan Cruz Maisu
Independent Researcher
Buenos Aires, Argentina
`github.com/asastuai`

April 2026

Abstract

We present Hermetic Computing, an interpretive framework that maps the Seven Hermetic Principles—attributed to Hermes Trismegistus and codified in the *Kybalion* (1908)—onto established concepts in cryptography and quantum computing. Each principle is expressed as a Rust trait and paired with a corresponding computational analogue: Mentalism with information as substrate, Correspondence with structure-preserving maps (group homomorphism), Vibration with frequency-domain analysis (DFT), Polarity with continuous duals (qubits, spectral logic), Rhythm with cyclic computation and timing analysis, Causality with provenance systems, and Generation with generative-formative dynamics. We compose two illustrative cryptographic artifacts from these primitives: (1) the *Hermetic Hash*, a 256-bit hash function composed of seven alchemical stages, with a measured avalanche ratio of 0.5001 on the test set; and (2) the *Magnum Opus*, a stream cipher in which an arbitrary "intent" byte string is mixed into the key schedule, producing distinct keystreams for distinct intents. Both artifacts use floating-point arithmetic internally and have not been subjected to formal cryptanalysis; they are educational demonstrations, not production primitives. The contribution of this paper is framing, not cryptographic: it offers a vocabulary for reading existing cryptographic concepts through a hermetic lens, along with executable Rust illustrations. Rigorous formalizations of purpose-bound encryption exist in Attribute-Based and Functional Encryption; this work is adjacent to those lineages, not a replacement. An accompanying SDK is published at `crates.io/crates/intent-cipher`.

Keywords: hermetic principles, homomorphic encryption, spectral logic, frequency-domain hashing, philosophy of computing

1 Introduction

1.1 Thesis

The Seven Hermetic Principles, formulated millennia before the invention of computers, can be read as structurally analogous to a range of modern cryptographic and quantum computing concepts. We demonstrate this reading not as proof but through implementation: each principle is formalized as a Rust trait, composed into illustrative artifacts, and tested for correctness. The mapping is interpretive; the code makes the analogy concrete enough to read and contest.

1.2 Background

The Hermetic tradition traces to Hermes Trismegistus, a syncretic figure combining the Greek Hermes with the Egyptian Thoth. The core teachings were transmitted through the *Corpus Hermeticum* (1st–3rd century CE), the *Emerald Tablet* (late 8th–early 9th century CE) [1], and the *Kybalion* (1908) [2], which codified seven principles: Mentalism, Correspondence, Vibration, Polarity, Rhythm, Cause and Effect, and Generation.

These principles have been studied in philosophical and esoteric contexts but, to our knowledge, have not been expressed as executable computational primitives with direct computational analogues. This paper offers such an expression.

1.3 Prior Art

Tavares (2020) [3] applied hermetic axioms to software architecture without cryptographic application. Homomorphic encryption [4] and post-quantum cryptography [5] are well-established fields with no connection to hermetic philosophy. Morgan (2018) drew parallels between hermeticism and quantum physics without computational implementation. We are not aware of prior work expressing the seven hermetic principles as executable computational primitives in the form presented here.

1.4 Contributions

1. A Rust implementation of the seven hermetic principles as traits, with each paired to an established computational analogue.
2. Two illustrative cryptographic artifacts composed from those traits—a 256-bit hash (Hermetic Hash) and a stream cipher (Magnum Opus)—intended as pedagogical demonstrations, not production primitives.
3. A set of conceptual correspondences between hermetic vocabulary and existing computational concepts, each accompanied by executable code.

1.5 Positioning and Scope

This paper is a framing contribution, not a cryptographic one. The implementations use floating-point arithmetic and have not been subjected to formal cryptanalysis. They are appropriate as educational artifacts, illustrations, or entries in an interdisciplinary dialogue. They are not appropriate as production cryptography. See Section 7 for a detailed discussion of limitations.

2 The Seven Principles as Computational Primitives

2.1 Mentalism — The Information Axiom

"The All is Mind; the Universe is Mental." — The Kybalion, Ch. VII

Every computational entity can be reduced to information (Essence) and reconstructed from it. We define three planes of manifestation: Spiritual (pure potential), Mental (conceptual), and Physical (measured). The trait `Mentalism` requires `to_essence()`, `from_essence()`, and `plane()`. Corrupted information (NaN) cannot manifest—`from_essence` returns `None`.

2.2 Correspondence — The Homomorphism Principle

"As above, so below; as below, so above." — The Emerald Tablet [1]

A structure-preserving map between two domains. The trait `Correspondence<Above, Below>` defines `ascend` (below → above) and `descend` (above → below) with an associated `Gap` type representing information lost in translation.

An Additive Correspondence. For an additive correspondence h:

$$h(a) + h(b) = h(a + b) \tag{1}$$

This is the defining property of a group homomorphism, and matches the additive part of homomorphic encryption [4]. The Principle of Correspondence mirrors the defining property of group homomorphism in its additive form.

We implement three correspondences: Hebrew Gematria (word → number, a pre-modern hash-like mapping where descent requires context), Additive Homomorphism (structure-preserving), and Composed Correspondence (demonstrating transitivity across multiple planes).

2.3 Vibration — The Frequency Principle

"Nothing rests; everything moves; everything vibrates." — The Kybalion, Ch. IX

All information has a frequency signature. We implement a Discrete Fourier Transform that decomposes byte sequences into their constituent frequencies. The roundtrip data → spectrum → data is verified lossless. The DFT reveals hidden structure: a repeating pattern `[0,255,0,255...]` yields a dominant frequency of 0.50.

Technical finding: Perfect reconstruction requires all N frequency bins, not the Nyquist half. The full vibrational picture is required for materialization.

2.4 Polarity — The Spectral Logic Principle

"Everything is dual; opposites are identical in nature, but different in degree." — The Kybalion, Ch. X

Every type has a dual. Opposites are not boolean but extremes of a continuous spectrum. We define `Polarity` with `pole_position()` returning a value in $[0, 1]$ and `transmute()` moving along the spectrum.

Observation. The qubit naturally satisfies the `Polarity` trait. $|0\rangle$ is the negative pole, $|1\rangle$ the positive pole, superposition is the spectrum between them, and the Bloch sphere [6] is a polar spectrum in 3D. The Hadamard gate is one involution among many that match the alchemical *Solve et Coagula* pattern: $H|0\rangle = |+\rangle$ (dissolve) and $H|+\rangle = |0\rangle$ (reconstitute), with $H^2 = I$ (self-inverse).

Observation. $|+\rangle$ and $|-\rangle$ have identical measurement probabilities but different global phase. Polarity and phase can be varied independently—standard quantum mechanics, usefully read as two "axes" rather than one.

2.5 Rhythm — The Cyclic Computation Principle

"Everything flows, out and in; the pendulum-swing manifests in everything." — The Kybalion, Ch. XI

All computation is cyclical. The hermetic Law of Neutralization—the master dampens the swing—maps directly to constant-time execution, where timing variance is eliminated to prevent side-channel attacks [7]. Period detection via autocorrelation is the classical analogue of Shor's period-finding [8], which uses the Quantum Fourier Transform to break RSA.

2.6 Causality — The Provenance Principle

"Every Cause has its Effect; every Effect has its Cause." — The Kybalion, Ch. XII

Every value carries its complete causal history. The `Causal<T>` type wraps any value with origin (Prima or Derived), parent IDs, and depth. The `CausalChain` records and traces full lineage—structurally analogous to a blockchain where each block is causally linked to its predecessor [9].

2.7 Generation — The Creative Computation Principle

"Gender is in everything; everything has its Masculine and Feminine Principles." — The Kybalion, Ch. XIV

Creation requires two complementary forces: generative (projects possibilities) and formative (selects and shapes). The `Generation` trait defines `generate()` and `form()`. We implement `KeyForge` for cryptographic key generation and `EmergentNumber` where composition can produce properties absent from parts: `emerge`$(2, 2) = 4$ gains the property `perfect_square`, a compact illustration of a phenomenon long studied in complexity theory and systems science.

3 The Hermetic Hash

A 256-bit hash function composed through seven alchemical stages:

Stage	Operation	Principle	Function
I	Calcination	Mentalism	Normalize input to 64 samples
II	Dissolution	Vibration	DFT into 64 frequency bins
III	Separation	Polarity	Split into magnitude and phase
IV	Conjunction	Generation	Non-linear marriage of opposites
V	Fermentation	Rhythm	7 rounds, prime reaches [1,7,13,19,29,37,43]
VI	Distillation	Correspondence	Fold 64→32 via symmetric map
VII	Coagulation	Causality	Crystallize via XOR-fold of f64 bits

Table 1: The seven alchemical stages of the Hermetic Hash.

Results (50 inputs, ∼100 bit-flip tests each):

- Avalanche ratio: **0.5001** (ideal: 0.5000, deviation: 0.015%)
- Byte distribution: $\chi^2 = 244.26$, 0 missing values out of 256
- Collision resistance: 0 collisions in 1000 sequential inputs
- Single-bit sensitivity: 52% of output bits changed (ideal: 50%)

The avalanche ratio satisfies the Strict Avalanche Criterion [7] within statistical noise on the measured sample.

Caveat. These metrics are single-platform. The internal use of floating-point arithmetic (DFT, sin, cos, f64 bit extraction at the coagulation stage) precludes bit-identical cross-platform output. The hash is appropriate as an illustration of how seven stages can compose into a mixing function; it is not appropriate as a production hash.

4 The Magnum Opus: An Intent-Keyed Stream Cipher

A stream cipher whose keystream is generated from the interaction of all seven principles. It takes two inputs: a **key** and an **intent** (an arbitrary byte string).

4.1 Architecture

Seven internal state components (one per principle) contribute to each output byte:

1. **Essence** (Mentalism): key + intent reduced to $\mathbb{R}^{32}$
2. **Veil** (Correspondence): structure-preserving transform
3. **Spectrum** (Vibration): DFT-derived frequency state with feedback
4. **Poles** (Polarity): spectral positions evolving per byte
5. **Rhythms** (Rhythm): 32 oscillators with prime periods $\{2, 3, 5, \ldots, 131\}$
6. **Causal accumulator** (Causality): FNV-1a running hash of all output
7. **Seeds** (Generation): LCG expansion + formative masking

4.2 Property

Same key with different intent produces distinct keystreams. A matching intent decrypts correctly; a mismatched intent yields different bytes. This is the expected behaviour of any cipher whose key schedule incorporates additional input.

4.3 Framing

We use the name *intent* for this input to emphasise a semantic use-case: an application that wants to distinguish encryption sessions by purpose may find this terminology legible. Functionally, however, `intent` behaves as additional key material—closer to a nonce or salt than to a semantic parameter. The cipher does not parse or evaluate the meaning of the intent string; it mixes its bytes into the key derivation.

4.4 Distinction from Rigorous Purpose-Bound Cryptography

Attribute-Based Encryption [10] formally binds access policy to decryption: a decryption key is valid only if it satisfies a policy over attributes encoded at encryption time. Functional Encryption [11] generalises this so that decryption reveals only a function of the plaintext. These constructions come with formal security definitions, proofs, and carefully designed algebraic machinery. The Magnum Opus does not belong to these lineages; it is a pedagogical demonstration of a semantic-nonce pattern, not a new cryptographic category.

4.5 Verified Properties

- Perfect roundtrip for all tested messages (up to 10KB)
- Wrong key: decryption fails (produces different bytes)
- Mismatched intent: decryption fails (produces different bytes)
- Keystream distribution: < 25 max count per byte value in 1000 bytes (> 200 unique values)
- Causal state: unique after every byte produced
- Deterministic on a single platform: same inputs always produce identical output

5 Conceptual Correspondences

Hermetic Concept	Computational Analogue	Status
Correspondence (additive)	Group homomorphism; additive HE [4]	Analogy via code
Gematria	Hash function (many-to-one)	Pedagogical mapping
Polarity spectrum	Qubit on Bloch sphere [6]	Trait satisfied
Solve et Coagula	Involutions (Hadamard: $H^2 = I$)	Pattern match
Vibration	Discrete Fourier Transform	Roundtrip implemented
Pendulum neutralization	Constant-time execution	Analogy
Period of rhythm	Shor's period-finding [8]	Analogy
Causal chain	Provenance / blockchain [9]	Structural analogue
Generative / Formative	GAN [12] / evolutionary algorithm	Analogy

Table 2: Conceptual correspondences between hermetic and computational concepts.

5.1 Observations

These are observations that emerged while building the code, offered as pedagogical framings rather than novel results:

1. **Polarity and phase are independent.** $|+\rangle$ and $|-\rangle$ have identical measurement probabilities but different global phase—a standard property of Hadamard-basis states, usefully read as two "axes" (polarity and vibration) varying independently.
2. **Composition can produce properties absent from parts.** `EmergentNumber::emerge`$(2, 2) = 4$ gains the property `perfect_square`, which neither input had—a compact illustration of a phenomenon long studied in complexity theory and systems science.
3. **Naming a cipher's extra input as "intent".** Framing a nonce-like parameter as *intent* is a UX and pedagogical choice that may help explain purpose-legible encryption to non-specialists. It is not a new cryptographic primitive.

6 Speculative Framings: Post-Quantum Cryptography

The following connections are offered as interpretive framings, not technical claims. They suggest vocabulary that may aid pedagogy or discussion; they do not constitute cryptographic contributions.

Lattice-based cryptography [5] depends on the hardness of finding short vectors in high-dimensional lattices. In hermetic terms, the lattice exists in a higher plane where operations are tractable, but descending (finding the short vector) requires information lost in the Correspondence Gap. The Learning With Errors problem [5] adds deliberate Vibration (noise) until the original frequency is unrecoverable. Shor's algorithm [8] breaks classical cryptography by finding hidden Rhythm through quantum Vibration—three principles composing to solve what classical computation cannot.

7 Limitations

- **Not formally cryptanalyzed.** The Hermetic Hash and Magnum Opus have been tested for correctness and basic quality metrics, not for security. They have not been subjected to differential, linear, or distinguishing analysis; keystream period has not been established; preimage and second-preimage resistance are not verified.

- **Floating-point internals.** Both artifacts use f64, transcendental functions (sin, cos), and a DFT. This precludes bit-identical output across platforms, compilers, and math libraries. A production primitive would require a full reimplementation on integer arithmetic.
- **Framing, not proof.** The connections drawn between hermetic principles and computational concepts are interpretive. They are meant to be read and discussed, not treated as mathematical theorems. The code makes the analogy concrete; it does not establish it formally.
- **Post-quantum connection is conceptual.** Section 6 offers vocabulary, not implementation.
- **Use audited libraries for real cryptography.** `ring`, `rustls`, `age`, `libsodium`, and other audited implementations should be preferred for any security-bearing application. The artifacts here are educational.

8 Conclusion

The Seven Hermetic Principles offer a compact interpretive vocabulary for concepts that cryptography and quantum computing already express in their own languages. Expressing them as Rust traits makes the correspondences concrete enough to read, discuss, and contest. The two illustrative artifacts—the Hermetic Hash and the Magnum Opus—are educational compositions of those traits; they are not proposed as production primitives. The contribution here is framing: a vocabulary that may be useful for teaching, for interdisciplinary dialogue, or for those who already think through hermetic categories and want a way to connect that thinking to computation. Whether the vocabulary proves durable is for readers and time to decide.

Reproducibility: All code and tests can be inspected and executed:

```
git clone https://github.com/asastuai/kybalion && cargo test
```

Accompanying SDK: The intent-keyed cipher is available as a zero-dependency Rust crate:

```
cargo add intent-cipher
```

Interactive demonstration: https://asastuai.github.io/kybalion/

References

[1] *The Emerald Tablet of Hermes Trismegistus.* In *Kitāb Sirr al-Khalīqa*, attr. Balīnas, late 8th–early 9th c. CE. Studied in Holmyard, E.J. "The Emerald Table." *Nature* 112:525–526, 1923.

[2] Three Initiates [W.W. Atkinson]. *The Kybalion.* Yogi Publication Society, 1908. Attribution in Deslippe, P. *The Kybalion: The Definitive Edition.* TarcherPerigee, 2011.

[3] Tavares, A.P. "Hermetic Principles of Computing." 2020.

[4] Gentry, C. "Fully Homomorphic Encryption Using Ideal Lattices." *STOC '09*, pp. 169–178, ACM, 2009. DOI: 10.1145/1536414.1536440.

[5] Regev, O. "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography." *STOC '05*, pp. 84–93, 2005. 2018 Gödel Prize.

[6] Bloch, F. "Nuclear Induction." *Physical Review* 70(7–8):460–474, 1946.

[7] Webster, A.F. and Tavares, S.E. "On the Design of S-Boxes." *CRYPTO '85*, LNCS 218, pp. 523–534, 1986. DOI: 10.1007/3-540-39799-X_41.

[8] Shor, P.W. "Algorithms for Quantum Computation." *FOCS '94*, pp. 124–134, IEEE, 1994. Journal: *SIAM J. Computing* 26(5):1484–1509, 1997.

[9] Nakamoto, S. "Bitcoin: A Peer-to-Peer Electronic Cash System." 2008.

[10] Sahai, A. and Waters, B. "Fuzzy Identity-Based Encryption." *EUROCRYPT 2005*, pp. 457–473.

[11] Boneh, D., Sahai, A. and Waters, B. "Functional Encryption: Definitions and Challenges." *TCC 2011*, pp. 253–273.

[12] Goodfellow, I.J. et al. "Generative Adversarial Nets." *NeurIPS 2014*, pp. 2672–2680.

[13] Shannon, C.E. "Communication Theory of Secrecy Systems." *Bell Syst. Tech. J.* 28(4):656–715, 1949.

[14] Holland, J.H. *Emergence: From Chaos to Order.* Addison-Wesley, 1998.

[15] Scholem, G. *Kabbalah.* Keter Publishing, 1974.